

Project Context for AI Agents

AI Context Document

Project Context for AI Agents

Use this file to onboard an AI coding assistant onto VelaDial. It is written as a technical handoff, not as marketing copy.

Project summary

VelaDial is a dual-node, silent, local-first smart lighting controller for a bedroom with multiple Tuya WiFi RGB bulbs.

The goal is to control the lights without voice commands, phone use, cloud latency, relay clicks, or bright screens at night.

The system has three parts:

1. **Door-side controller** — ELECROW 1.28 in ESP32-S3 rotary touch display running ESPHome and LVGL. Includes TSL2591 ambient light sensor and SHT45 temperature/humidity sensor.
2. **Bedside controller** — ESP32-C6 with APDS-9960 gesture/proximity sensor and VL53L4CD time-of-flight distance sensor, running ESPHome.
3. **Home Assistant hub** — Raspberry Pi running Home Assistant with local Tuya bulb control.

Current build goal

Write production-ready ESPHome YAML for v1 only:

1. **Door-side rotary display** — Multi-theme firmware engine containing 20 selectable UI themes. The main control UI remains 3 pages (Power, Brightness, Presets) plus a hidden Theme Selector accessed via long-press. Behavior is locked as described in this file and in [docs/03_App_Flow.md](#) and [docs/04_UI_UX_Design_Brief.md](#).
2. **Adaptive display brightness** — TSL2591 lux drives display backlight PWM locally on the ESP32-S3, no Home Assistant round-trip.
3. **Bedside controller v1:**
4. APDS-9960 standalone left/right gestures.

5. VL53L4CD standalone hand-hold nightlight, only if VL53L4CD ESPHome support is verified.
6. **Sensor fusion is v2 / future, not v1.**
7. **ELECROW pinout must be validated** against the actual board silkscreen / PCB revision before production firmware. Multiple board revisions exist.

The YAML should be readable, commented, configurable, and safe for public source control.

Main design constraints

- Silent interaction only.
- Local-first control path. Internet / ISP outage alone should not break normal control if local LAN, Raspberry Pi, Home Assistant, and LocalTuya are healthy.
- No cloud dependency for normal control.
- Target response below 500 ms on the local network.
- Dark UI with low light pollution.
- Large readable labels for a 240 × 240 round display.
- Keep credentials and local keys out of the repository.
- Main control UI is exactly **3 pages**: Power, Brightness, Presets. (Plus a hidden Theme Selector).
- Temperature/humidity is secondary diagnostics only, not a required main page.
- Bedroom safety is more important than shortcut speed; a second deliberate action is what acts.

Hardware already selected

Door-side controller

- ELECROW CrowPanel 1.28 in ESP32 rotary display.
- ESP32-S3R8 class board.
- 240 × 240 round IPS display.
- GC9A01A display over SPI.
- CST816D capacitive touch over I2C.
- Physical rotary encoder with press.
- 5-LED WS2812 ambient ring.
- 16 MB flash and 8 MB PSRAM on the purchased model.
- TSL2591 ambient light sensor (I2C address `0x29`).
- SHT45 temperature/humidity sensor (I2C address `0x44`).

Bedside controller

- Adafruit ESP32-C6 Feather or equivalent ESP32-C6 board.
- APDS-9960 gesture/proximity sensor over I2C (address `0x39`).
- VL53L4CD time-of-flight distance sensor over I2C (address `0x29`).
- Intended to be mounted in a small dark enclosure near the bed.
- Optional bedside **status LED** (single-color amber or red, any free GPIO + current-limiting resistor) for silent local input acknowledgement — see "Bedside status LED behavior (optional, v1)" below.

Hub and lights

- Home Assistant runs on a Raspberry Pi.
- The target room has five Tuya WiFi RGB bulbs.
- Bulbs should be grouped in Home Assistant as one controllable light group.
- LocalTuya or an equivalent local Tuya LAN integration is the preferred control path.

Sensor-to-node assignment

NODE	ESP32	SENSORS	I2C ADDRESSES
Door-side	ESP32-S3 (ELECROW rotary display)	TSL2591, SHT45	<code>0x29</code> , <code>0x44</code>
Bedside	ESP32-C6	APDS-9960, VL53L4CD	<code>0x39</code> , <code>0x29</code>

I2C architecture

Door-side and bedside nodes use separate I2C buses. They are independent ESP32 devices on the same WiFi network, each with their own I2C peripheral.

TSL2591 and VL53L4CD both use `0x29`, but this is not a conflict because they are on different ESP32 nodes. Each node's I2C bus only has one device at `0x29`.

No XSHUT pin reassignment or TCA9548A I2C multiplexer is required for the first-build architecture. They remain as future fallback only if the architecture ever places both `0x29` sensors on the same bus.

Decisions already made

- Door-side controller: ELECROW 1.28 in rotary display.
- Bedside controller: ESP32-C6 plus APDS-9960 plus VL53L4CD.
- Firmware: ESPHome for both controllers.

- UI framework: ESPHome LVGL.
- Hub: Home Assistant on a Raspberry Pi.
- Bulb control: local Tuya LAN control through Home Assistant + LocalTuya.
- Communication: ESPHome Native API to Home Assistant, then Home Assistant to the light group.
- TSL2591 controls display/backlight only, not room bulbs.
- SHT45 is secondary diagnostics, not a required main page.
- VL53L4CD enables deliberate nightlight hold, not accidental triggers.
- **No required 4th Environment page on the display.**
- **Main control UI is exactly 3 pages: Power, Brightness, Presets.** (Plus a hidden Theme Selector).
- **First-build presets are locked to exactly 4: Warm White, Soft Amber, Neutral White, Low Nightlight.** Optional RGB accent is v2 / future only.
- **Bedside v1 uses APDS-9960 standalone gestures + VL53L4CD standalone hand-hold nightlight.** Sensor fusion is v2 / future only.
- **VL53L4CD ESPHome support remains unverified** — see "VL53L4CD support truth" below.

ELECROW rotary display pinout

Display — GC9A01A over SPI

FUNCTION	GPIO
SCLK	10
MOSI	11
MISO	not used
DC	3
CS	9
RST	14
Backlight	46

Touch — CST816D over I2C

FUNCTION	GPIO
SDA	6
SCL	7

FUNCTION	GPIO
INT	5
RST	13

Rotary encoder

FUNCTION	GPIO
Encoder A	45
Encoder B	42
Switch press	41

RGB LED ring — WS2812

FUNCTION	GPIO / VALUE
Data	48
LED count	5

ESPHome notes for ELECROW display

- Board: `esp32-s3-devkitc-1`.
- Framework: ESP-IDF.
- Display model: `GC9A01A` under the `ili9xxx` platform.
- Touch platform: `cst816`, address `0x15`.
- `invert_colors: true` may be required.
- PSRAM and flash settings should match the board revision.
- Touch transform and encoder direction must be verified on real hardware.

Locked door-side UI behavior

The following behaviors are locked for the first build. They are owned by `docs/03_App_Flow.md` and `docs/04_UI_UX_Design_Brief.md`; the lines below are a concise summary for AI handoff.

Pages

- **Main control UI is exactly 3 pages: Power, Brightness, Presets.** (Plus a hidden Theme Selector).
No required 4th Environment page.

Page navigation

- Horizontal swipe navigates between pages.
- Left swipe = next page (Power → Brightness → Presets → Power).
- Right swipe = previous page.
- A small **3-dot page indicator** appears near the bottom of the round screen.
- The active page's dot is amber; inactive dots are low-contrast white.

Wake behavior (locked)

- **Touch while asleep:** wake only.
- **Knob rotation while asleep:** wake only.
- **Knob press while asleep:** wake only.
- A second deliberate action while the display is awake is what toggles or sends a command.
- Bedroom safety is more important than shortcut speed.

Per-page knob press behavior (applies only when display is awake)

- **Power page:** knob press toggles the bedroom light group (same as tapping the center).
- **Brightness page:** knob press returns to the Power page.
- **Presets page:** knob press applies the currently highlighted preset.
- **Theme Selector (any page):** long-press (>1.5s) enters the Theme Selector.
- **While asleep:** knob press wakes only.

Page content rules

Power page

- Large central power control.
- Tap to toggle the bedroom light group.
- Knob press (when awake) also toggles the bedroom light group.
- Optional WS2812 LED ring glows amber when lights are on.
- Status text: On / Off / Unavailable.

Brightness page

- Circular arc around the edge of the round display.

- Large center percentage label.
- Rotary knob changes brightness in about 5 % steps.
- Commands should be debounced enough to avoid flooding Home Assistant.
- Knob press returns to the Power page.

Presets page

- Exactly 4 first-build presets:
- Warm White
- Soft Amber
- Neutral White
- Low Nightlight
- 2×2 large touch targets on the 240×240 round screen.
- Active preset uses an amber border (or equivalent amber highlight).
- Tap a preset to apply.
- Knob rotation cycles the preset highlight; knob press applies the highlighted preset.
- Optional RGB accent preset is **future / v2 only**.
- **No dense color picker, color wheel, or per-bulb editor in first build.**

"Unavailable" UI wording

When Home Assistant, the Raspberry Pi, or the local LAN is unreachable:

- Use **"Unavailable"** as the preferred door-side UI wording.
- Show a subtle persistent badge or small label, not a full-screen overlay.
- Do **not** pretend a command succeeded without state confirmation from Home Assistant.
- Dim or desaturate affected controls if useful for clarity.
- **No** flashing, popup modals, audio, or full-screen aggressive errors.
- Recover automatically when the connection returns. No user action required.

Adaptive display brightness

TSL2591 measures ambient lux on the door-side node and maps it to display backlight PWM.

- Bright room: higher backlight for readability.
- Dim room: moderate backlight.
- Dark bedroom: very low backlight to avoid disturbing sleep.
- Transitions are smooth, not flickery.

- Night brightness is capped even after wake interaction.
- TSL2591 controls display/backlight only, not room bulbs.
- This runs locally on the ESP32-S3 without Home Assistant round-trip.

UI design rules

- Background: pure black, `#000000`.
- Primary accent: warm amber, `#FFB300`.
- Text: white, `#FFFFFF`.
- Large primary font size, roughly 48 px or larger where practical.
- Flat design, no heavy shadows.
- Screen dims to about 10 % after 60 seconds of inactivity.
- Touch or knob movement wakes the screen (wake-only first per "Wake behavior" above).
- Waking the screen does not toggle the lights; that takes a second deliberate action.

Bedside controller v1

The bedside device is headless and touchless. v1 is **APDS-9960 standalone gestures + VL53L4CD standalone hold**. No sensor fusion in v1.

v1 — APDS-9960 standalone gestures

- **Wave left:** turn the bedroom light group off.
- **Wave right:** turn the bedroom light group on.
- About 1 s cooldown after each successful gesture to prevent repeat-fire.
- Unclear, ambiguous, or noisy gesture reads are ignored. APDS-9960 misses are expected in real-world use.
- Mounting and ambient lighting (sunlight, IR sources) affect reliability.

v1 — VL53L4CD standalone hand-hold nightlight

Only if VL53L4CD ESPHome support is verified (see "VL53L4CD support truth" below).

- **Trigger:** hand stable in the 5–10 cm range above the bedside sensor.
- **Hold duration:** continuous in-range for about 1.5 s.
- **Hand-near debounce:** require the hand to be in range for at least about 200 ms before the 1.5 s hold timer starts. Prevents quick pass-by motion from starting the timer.
- **Cooldown:** about 5 s after a successful trigger before another nightlight trigger is possible.

- **Action on trigger:** Home Assistant turns the bedroom group on at a low warm nightlight (about 10 % brightness, warm color).
- **Quick pass-by motion must not trigger.**

v2 / future — sensor fusion (not required for first firmware)

- VL53L4CD distance reading could be used to arm, wake, or refine APDS-9960 gesture detection — for example, only accepting APDS-9960 gesture events while VL53L4CD reports a hand within an arming range.
- Sensor fusion is **not required for the first firmware build**. It should be added only after both v1 standalone paths are verified to work reliably in the actual bedside mounting.
- See docs/08_Sensor_Expansion_Research.md for the broader fusion design discussion.

General bedside rules

- No audio, no voice, no beeps.
- Cooldowns remain active even during Home Assistant failure to prevent command spam.

Bedside status LED behavior (optional, v1)

An optional small status LED on the bedside controller provides silent local feedback. This is optional first-build hardware, not required.

- **1 blink:** input detected (gesture or button registered locally by the ESP32).
- **2 blinks:** nightlight hold triggered.
- The status LED does **not** confirm bulb state. Bulb state is shown by the bulbs themselves.
- v1 does **not** use a steady or pulsing offline-status LED. A constantly lit LED in a dark bedroom causes light pollution, which contradicts the project's bedroom-safe principle.
- Offline / unavailable status belongs primarily on the door-side display, not on a constantly lit bedside LED.

VL53L4CD support truth

- VL53L4CD remains the **planned** bedside ToF sensor because it has been purchased.
- VL53L4CD ESPHome support path is **unverified**.
- Do not assume native ESPHome support.
- Do not assume VL53L4CD is definitely custom-only unless verified.
- **VL53L0X is the verified ESPHome-supported ToF fallback only.** Do not switch to VL53L0X unless VL53L4CD validation blocks progress and the owner approves.

- Do not describe **VL53L1X** as a native ESPHome fallback unless current official ESPHome documentation or a trusted component source proves it. (Live verification at the time of writing showed neither a docs page nor a component in the ESPHome dev branch.)

Environmental data truth

- SHT45 temperature/humidity is **secondary**.
- There is **no required Environment page** on the device.
- Do not put temperature or humidity on Power / Brightness / Presets as a required UI element.
- A diagnostics view, idle mini-status, or long-press info overlay are **future / v2 possibilities only, not first build**.
- SHT45 values are exposed to Home Assistant and can be shown in the Home Assistant dashboard.
- **No HVAC / comfort automation in first build.**

Network reliability truth (concise)

VelaDial is local-first. Detailed deployment guidance lives in [docs/02_TRD.md](#); the lines below are the short version for AI handoff.

- **Internet / ISP outage alone should not break normal control** if local LAN, Pi, Home Assistant, and LocalTuya are healthy. No cloud dependency for normal control.
- **LAN / WiFi / router / AP instability is the real network risk.** Local-first does not protect against LAN-side failure.
- Recommended deployment (full list in TRD):
- Raspberry Pi on Ethernet to the router if possible.
- UPS for both the Raspberry Pi **and** the router / WiFi AP / network gear (Pi-only UPS is not enough if the network gear loses power through the same blip).
- Stable 2.4 GHz IoT SSID for Tuya bulbs and ESP32 devices.
- Static DHCP leases for Pi, ESP32 nodes, and each Tuya bulb.
- A manual wall switch on the bulb circuit remains the only simple true "everything is broken" fallback. It lives at the wall, not in the firmware.

Recommended Home Assistant entity model

Keep this configurable in YAML substitutions:

- `light.bedroom_group` — group of the five bedroom bulbs.

Do not hardcode private device keys, tokens, passwords, or local environment values.

What another AI should do next

The order matters. Do **not** jump into production firmware before validation steps.

1. **Read first.** Read `README.md`, this file, and all files in `docs/` — especially `docs/02_TRD.md`, `docs/03_App_Flow.md`, `docs/04_UI_UX_Design_Brief.md`, and `docs/08_Sensor_Expansion_Research.md`. These are the locked source of truth; this file is a summary of them.
2. **Validate hardware before production firmware.**
3. Validate the ELECROW board revision and pinout against `hardware/elecrow_pinout.md` and the actual board silkscreen. Multiple ELECROW CrowPanel 1.28 in revisions exist.
4. Run an I2C scan on both nodes and confirm expected addresses (door-side `0x29`, `0x44`; bedside `0x39`, `0x29`).
5. **Verify VL53L4CD ESPHome support path** before writing hold-nightlight firmware. If support is blocked, do not silently substitute VL53LOX — surface the decision to the owner first.
6. **Door-side firmware candidate delivered (PR #56).** `esphome/door_side_rotary.yaml` is now a compile-passing multi-theme firmware candidate with 20 selectable themes, 3-page LVGL control flow (Power, Brightness, Presets), hidden Theme Selector via long-press, the 4 locked presets, page-swipe navigation, per-page knob press behavior, wake-only-first behavior, and "Unavailable" badge for HA / LAN outage. Compile PASSED (0 errors). **Hardware validation pending. Not physically validated. Further visual refinement may follow hardware testing.**
7. **Next steps are hardware validation.**
8. Flash the production door-side firmware to the ELECROW board and verify display, touch, encoder, backlight, TSL2591, SHT45, LED ring, and all 20 themes on real hardware.
9. Bedside bring-up: APDS-9960 standalone first. VL53L4CD second, gated on the support-path verification. **Do not implement sensor fusion in v1.**
10. **Constraints to honor.**
11. No cloud dependency for normal control.
12. No required 4th Environment page.
13. No dense color picker, color wheel, or per-bulb editor in v1.
14. No steady / pulsing bedside offline LED in v1 (bedroom light pollution).
15. No relay or mains-control hardware on the ESP32 in v1.
16. Mark any hardware assumption that still needs real-device validation.

Useful references

- ELECROW CrowPanel 1.28 in HMI ESP32 rotary display documentation.

- ESPHome LVGL documentation (esphome.io/components/lvgl/).
- ESPHome ili9xxx display documentation (covers GC9A01A).
- ESPHome CST816 touchscreen documentation.
- ESPHome rotary_encoder documentation.
- ESPHome APDS9960 documentation.
- ESPHome TSL2591 documentation.
- ESPHome SHT4x documentation.
- ESPHome VL53L0X documentation (fallback reference only; not the selected sensor).
- ST VL53L4CD datasheet and ESPHome community notes (support path unverified at time of writing).
- Home Assistant LocalTuya documentation and community notes.
- SmartKnob-style rotary interaction patterns as inspiration, not as code to copy.